DO NOW — RECALL

Recursion: call stack and tracing

Name: _____ Class: _____ Date: _____

1. Factorial(n) returns 1 when n=0; otherwise it returns n*Factorial(n-1). Trace the successive argument values in Factorial(3), stopping when the base case is reached.

2. What is the difference between a function calling another function and returning a value to its caller?

3. A stack contains bottom to top P, Q, R. After one pop, what is at the top?

4. A function Square(x) returns x*x. Find Square(2)+Square(3), showing both calls.

DO NOW — CHECK / OBJECTIVES

Correct and improve

Check page 1 in GREEN PEN. Write corrections in full.

1. Model answer

3, 2, 1, 0. The n=0 call is the base case.

2. Model answer

A call transfers control into a function; return finishes that invocation and gives its result to the caller.

3. Model answer

Q. The topmost R is removed first.

4. Model answer

Square(2)=4 and Square(3)=9; the total is 13.

Vocabulary — know these terms

stack frame — An entry containing the state for one active function call.

unwind — Return from calls in reverse order after a base case is reached.

local variable — A variable owned by a particular invocation.

call stack — Last-in, first-out storage for active calls.

base case — A condition that ends recursion.

NEW LEARNING

Read, annotate and explain

Build the stack

SumTo(4) waits for SumTo(3), which waits for SumTo(2), which waits for SumTo(1).

Separate the local values

Each stack frame holds its own n: 4, 3, 2 or 1. The values do not overwrite one another.

Unwind from the top

SumTo(1) returns 1; SumTo(2) returns 3; SumTo(3) returns 6; SumTo(4) returns 10.

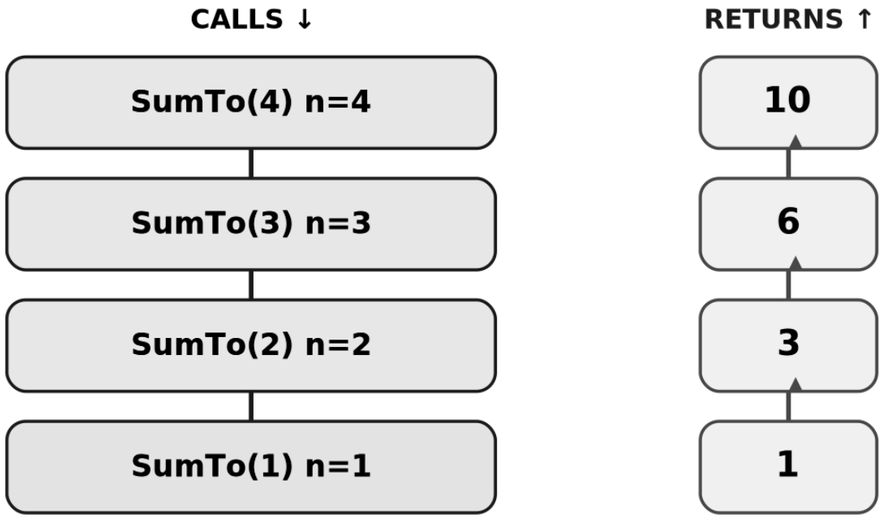
Explain one step in your own words, using the correct technical terms.

New Learning · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

What is the program actually doing?

This version accepts positive integers only ; zero and negative input are rejected. Each call has its own n . The next call is placed on top of the previous one.



Recursive calls moving downward and return values moving upwards. SumTo(4) n=4, SumTo(3) n=3, SumTo(2) n=2, SumTo(1) n=1.

WE DO — SOURCE CHECK / IMPROVE

Only use this section AFTER attempting the We Do, You Do and Exit Ticket. Compare with the original classroom lesson's CHECK explanation. Correct errors in GREEN PEN and explain why.

Guided 1 — original CHECK

Top frame

SumTo(1) is on top immediately before the first return.

Returns

1, 3, 6, 10, 15. SumTo(5) returns 15.

IMPROVE: Explain one correction to your own reasoning.

EXIT TICKET / REFLECTION

Check your understanding

Complete the actual lesson Exit Ticket before checking the answer.

Every recursive call uses the same local-variable storage.

How confident are you? Circle one: Confident / Getting there / Need help

Explain your choice, identifying one concept you can now use independently.

GREEN PEN — Correct and explain

Choose one mistake from your independent work. Write the correct method and explain precisely why your original answer was wrong.

Next step: revisit the model or a relevant revision-guide section if needed.

I DO — WORKED EXAMPLE

Follow the worked method

Draw the call-stack frames for SumTo(3). Which call returns first?

Push

The frames are SumTo(3), SumTo(2), then SumTo(1), with SumTo(1) at the top.

Stop

SumTo(1) is the base case; it returns 1 first.

Unwind

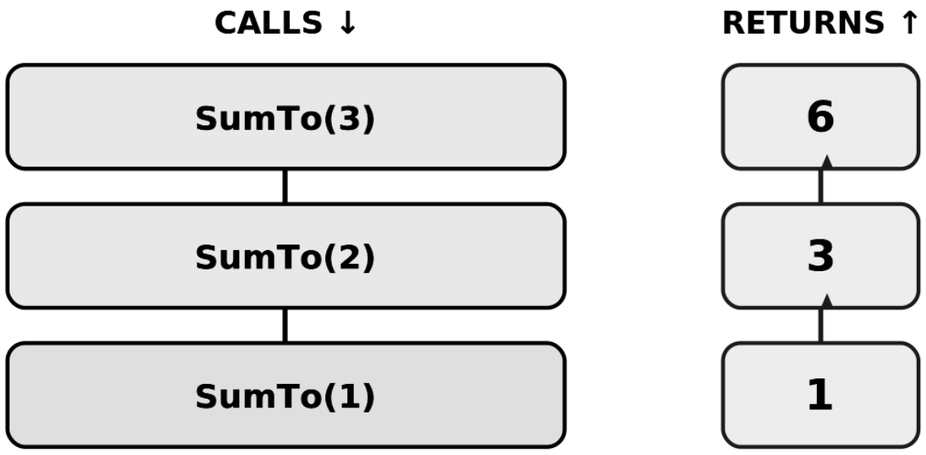
SumTo(2)=2+1=3, followed by SumTo(3)=3+3=6.

I Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Draw the call-stack frames for SumTo(3). Which call returns first?

return n + SumTo(n - 1);



Call trace and return trace: SumTo(3); SumTo(2); SumTo(1). Return values: 6, 3, 1.

YOU DO — INDEPENDENT PRACTICE

Questions · Part 2

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 3

A pupil writes that SumTo(4) returns 4+3+2+1 before SumTo(1) has executed. Explain the order error.

YOU DO — INDEPENDENT PRACTICE

Questions · Part 1

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 1

List the call-stack frames from bottom to top for SumTo(4).

Question 2

Trace the returned values for SumTo(6).

WE DO — GUIDED PRACTICE

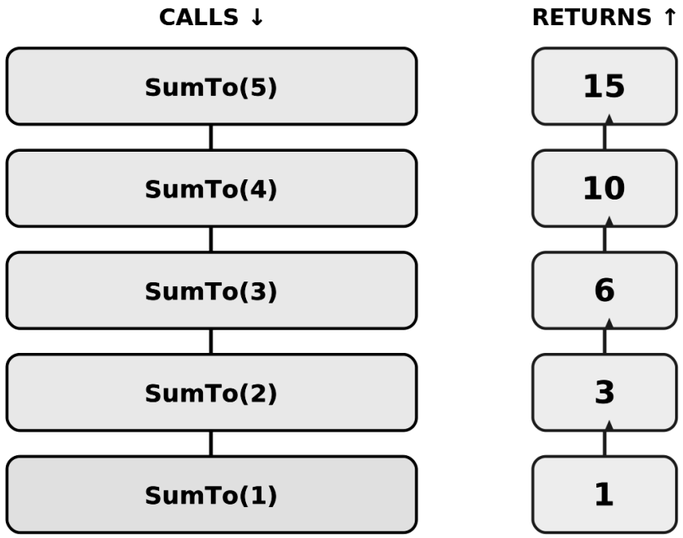
Try → Discuss → Improve

We Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Trace SumTo(5). Label the top of the stack immediately before the first return.

5 → 4 → 3 → 2 → 1



Call trace and return trace: SumTo(5); SumTo(4); SumTo(3); SumTo(2); SumTo(1). Return values: 15, 10, 6, 3, 1.

WE DO — TRY · DISCUSS · IMPROVE

TRY: Use the original guided questions and diagrams in the preceding We Do pages. DISCUSS: Explain and challenge methods before independent practice. IMPROVE: Note one change now; compare with the source model only AFTER the Exit Ticket.

Guided 1: Call sequence SumTo(5) → SumTo(4) → SumTo(3) → SumTo(2) → SumTo(1).

DISCUSS — Compare your methods. Justify one important decision, identify one difference, and agree a specific improvement.

IMPROVE — Write one precise correction or stronger explanation in your own answer.